

Lab Exercise 1: Triggering Nested Loop Join

The screenshot shows a PostgreSQL query editor interface. At the top, the database is 'dvd_rental/postgres@PostgreSQL 18'. Below the toolbar, the query is as follows:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT c.first_name, c.last_name, r.rental_date
3 FROM customer c
4 JOIN rental r ON c.customer_id = r.customer_id
5 WHERE c.customer_id = 1;
```

Below the query, the 'Data Output' tab is selected, showing the 'QUERY PLAN' for the query. The plan consists of 14 rows of information:

	QUERY PLAN
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=2.635..10.626 rows=32.00 loops=1)
2	Buffers: shared hit=1 read=152 dirtied=1
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.024..0.026 rows=1.00 loop=1)
4	Index Cond: (customer_id = 1)
5	Index Searches: 1
6	Buffers: shared hit=1 read=2
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=2.607..10.587 rows=32.00 loops=1)
8	Filter: (customer_id = 1)
9	Rows Removed by Filter: 16012
10	Buffers: shared read=150 dirtied=1
11	Planning:
12	Buffers: shared hit=86 read=4
13	Planning Time: 5.556 ms
14	Execution Time: 10.658 ms

At the bottom of the interface, it shows 'Total rows: 14' and 'Query complete 00:00:00.082'.

Analysis Questions:

- **Identify the Roles:** Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?

>>> Outer table is 'customer', and Inner table is 'rental'

- **Why Nested Loop?** How many rows did the customer table return? Why does this make Hash Join "too expensive"?

>>> The 'WHERE c.customer_id = 1' filter reduces the Outer Table to exactly 1 row. Nested Loop is the best strategy for very small datasets.

>>> 'customer' table returned exact 1 row because of the filter 'WHERE c.customer=1'

>>> A Hash Join requires a "BUILD" phase to create a Hash Table in memory (RAM). Allocating memory and building a hash map for just a single row is highly inefficient and considered "too expensive".

- **Efficiency:** Look at loops in the Inner Table node. Why is it 1?

>>> Because the 'customer' table returned exact 1 row, 'rental' table was required to loop only 1 time

Lab Exercise 2: Triggering Hash Function

Query Query History

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT f.title, l.name as language
3 FROM film f
4 JOIN language l ON f.language_id = l.language_id;
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.085..0.784 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.037..0.483 rows=1000.00 l...
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.031..0.032 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.020..0.021 rows=6.00 l...
10	Buffers: shared hit=1
11	Planning:
12	Buffers: shared hit=334
13	Planning Time: 16.754 ms
14	Execution Time: 0.954 ms

Analysis Questions:

- **Identify Phases: Which table is being Hashed (Built)? Which table is being Probed?**

>>> The 'language' table is being hashed, the 'film' table is being probed.

- **Memory Check: Look at the Hash node. What is the Memory Usage?**

>>> Memory usage: 9kB

- **Hypothesis: What happens if you run SET work_mem = '64kB'; and run it again? (Try it!)**

>>> There is nothing happened. Because 9 kB is still smaller than 64kB, the Ram does it easily.

Lab Exercise 3: Triggering Merge Join

Query		Query History
7	FROM film f	
8	JOIN inventory i ON f.film_id = i.film_id	
9	ORDER BY f.film_id; -- Explicit sort requirement	
Data Output		Messages Notifications
SQL Showing rows: 1 to 19 Page No: 1 of 1		
QUERY PLAN text		
1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=8.795..10.756 rows=4581.00 loops=1)	
2	Merge Cond: (f.film_id = i.film_id)	
3	Buffers: shared hit=89 read=24, temp read=10 written=12	
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.025..0.548 rows=1000.00 loop...	
5	Index Searches: 1	
6	Buffers: shared hit=84 read=1	
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=8.759..9.513 rows=4581.00 loops=1)	
8	Storage: Memory Maximum Storage: 17kB	
9	Buffers: shared hit=5 read=23, temp read=10 written=12	
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=8.727..9.053 rows=4581.00 loops=1)	
11	Sort Key: i.film_id	
12	Sort Method: external merge Disk: 80kB	
13	Buffers: shared hit=5 read=23, temp read=10 written=12	
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.070..4.478 rows=4581.00 loops=1)	
15	Buffers: shared hit=2 read=23	
16	Planning:	
17	Buffers: shared hit=93 read=1	
18	Planning Time: 7.717 ms	
Total rows: 19		Query complete 00:00:00.084 CRLF Ln

Analysis Questions:

- **Structure:** Do you see a Sort node? Or did it use Index Scan to get sorted data for free?

>>> Do not see Sort node. Instead, using Index Scan because the 'film_id' column already has an index. So that the Merge Join gets the sorted input without an extra sorting operation.

- **Compare:** Reset the settings (RESET ALL;) and remove the ORDER BY. Does it switch back to Hash Join? Why?

>>> Yes, because, the optimizer calculates that building a Hash Table in RAM (Hash Join) has a lower total cost and is more efficient for joining these specific tables than a Merge Join.

Query Query History

```

1 RESET ALL;
2 EXPLAIN (ANALYZE, BUFFERS)
3 SELECT f.film_id, f.title, i.inventory_id
4 FROM film f
5 JOIN inventory i ON f.film_id = i.film_id;

```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

QUERY PLAN	
text	SQL query of data
1	Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.763..1.763 rows=4581.00 loops=1)
2	Hash Cond: (i.film_id = f.film_id)
3	Buffers: shared hit=113
4	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.014..0.201 rows=4581.00 l...
5	Buffers: shared hit=25
6	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.740..0.740 rows=1000.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 59kB
8	Buffers: shared hit=88
9	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.025..0.210 rows=1000.00 l...
10	Buffers: shared hit=88
11	Planning:
12	Buffers: shared hit=6
13	Planning Time: 0.323 ms
14	Execution Time: 1.926 ms

Total rows: 14 Query complete 00:00:00.080 CRLF

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

Query Query History

```

1 -- Force a Sequential Scan by disabling Index Scans
2 SET enable_indexscan = off;
3 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
4
5 -- Re-enable and compare the Cost and Time
6 SET enable_indexscan = on;
7 EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';

```

Data Output Messages Notifications

Showing rows: 1 to 7 Page No:

QUERY PLAN	
text	SQL query of data
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.019..2.628 rows=16044.00 loops=...
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3	Buffers: shared hit=150
4	Planning:
5	Buffers: shared hit=84 read=3
6	Planning Time: 17.407 ms
7	Execution Time: 3.245 ms

Query
Query History

```

1  -- Force a Sequential Scan by disabling Index Scans
2  SET enable_indexscan = off;
3  EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
4
5  -- Re-enable and compare the Cost and Time
6  SET enable_indexscan = on;
7  EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';

```

Data Output
Messages
Notifications

Showing rows: 1 to 7
Page No: 1

	QUERY PLAN
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.022..1.872 rows=16044.00 loops=...
2	Filter: (rental_date > '2005-05-24 00:00:00':timestamp without time zone)
3	Buffers: shared hit=150
4	Planning:
5	Buffers: shared hit=4
6	Planning Time: 0.243 ms
7	Execution Time: 2.728 ms

Task 2: Join Strategy Analysis

Query
Query History

```

1  EXPLAIN (ANALYZE, BUFFERS)
2  SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id
3  FROM customer c
4  JOIN rental r ON c.customer_id = r.customer_id
5  JOIN inventory i ON r.inventory_id = i.inventory_id;

```

Data Output
Messages
Notifications

Showing rows: 1 to 22
Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=150.55..545.57 rows=16044 width=25) (actual time=1.345..7.546 rows=16044.00 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=184
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=0.314..3.950 rows=16044.00 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=159
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.031..0.795 rows=16044.00...
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.233..0.234 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 39kB
11	Buffers: shared hit=9
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.016..0.116 rows=599.0...
13	Buffers: shared hit=9
14	-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=0.976..0.981 rows=4581.00 loops=1)

Total rows: 22
Query complete 00:00:00.080
CRLF

Query

Query History

```

1  ANALYZE customer, rental, inventory;
2
3  EXPLAIN (ANALYZE, BUFFERS)
4  SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id
5  FROM customer c
6  JOIN rental r ON c.customer_id = r.customer_id
7  JOIN inventory i ON r.inventory_id = i.inventory_id;

```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🔍

📥

📶

SQL

Showing rows: 1 to 22

Page No: 1

	QUERY PLAN
	text
1	Hash Join (cost=150.55..545.57 rows=16044 width=25) (actual time=3.107..18.594 rows=16044.00 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=184
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=0.387..9.615 rows=16044.00 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=159
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.055..1.937 rows=16044.00...
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.314..0.317 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 39kB
11	Buffers: shared hit=9
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.029..0.135 rows=599.00...
13	Buffers: shared hit=9
14	-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=2.643..2.660 rows=4581.00 loops=1)

The Optimizer strictly selected a Double Hash Join strategy. Since there are no WHERE filters reducing the dataset, the system must join the entire tables. Hash Join is the most cost-effective method for large, unsorted bulk operations.

The execution is highly efficient because the Hash Tables fit perfectly into RAM (Batches: 1, Memory Usage: 39kB), completely avoiding any slow "Spill to Disk" bottlenecks.